FICHA DE IDENTIFICACIÓN DE TRABAJO DE INVESTIGACIÓN

Título	Calidad de generalización en redes neuronales discretas y continuas aplicadas a la predicción comercial en Oruro.	
Autor/es	**Nombres y Apellidos**	**Código de estudiantes**
	Emilio Gabriel Fernández Jiménez	88078
Fecha	31/03/2026	

Carrera	Ing. De Sistemas
Asignatura	Redes Neuronales
Grupo	A
Docente	Ing. Roger Rolando Perez Rojas
Periodo Académico	I/2026
Subsede	Oruro

Copyright © (2026) por (Emilio Gabriel Fernández Jiménez). Todos los derechos reservados.

Tabla De Contenidos

Lista De Tablas 3
Lista De Figuras 4
Introducción 5
Capítulo 1. Planteamiento del Problema 7
1.1 Formulación del Problema 7
1.2 Objetivos 7
1.3 Justificación 8
1.4 Planteamiento de Hipótesis 8
Capítulo 2. Marco Teórico 10
2.1 Área de Estudio/Campo de Investigación 10
2.2 Desarrollo del Marco Teórico 10
2.2.1 Evaluación de la Calidad de Generalización en Redes Neuronales Discretas vs. Continuas 10
2.2.2 Diferencias Arquitectónicas y Computacionales 11
2.2.3 Evaluación de la Generalización 13
2.2.4 Ventajas y Desventajas 15
2.2.5 Aplicaciones Prácticas 18
2.2.6 Estudios comparativos 20
2.2.7 Tendencias Emergentes 22
2.2.8 Herramientas, Framaworks y Protocolos Experimentales 24
Capítulo 3. Resultados y Discusión 27
3.1 Funciones de Soporte Vitales para el Método Científico 29
3.2 Codificación 30
3.3 Aplicación 40
Capítulo 4. Conclusiones 41
Referencias 43

Lista De Tablas

Tabla 1 – Comparaciones de Arquitecturas y Funciones de Activación 21

Lista De Figuras

Figura 1 - Evaluación de la Calidad de Generalización en Redes Neuronales Discretas vs. Continuas 11
Figura 2 - Ventajas y Desventajas 16
Figura 3 - Resultado 28
Figura 4 - Gráfico 29
Figura 5 - Aplicación 40

Introducción

En la actualidad, la aplicación de la Inteligencia Artificial (IA) y, específicamente, de las redes neuronales, ha revolucionado la forma en que las empresas gestionan sus operaciones comerciales. La predicción de demanda de inventario se ha convertido en una herramienta fundamental para optimizar el stock, evitar la escasez de productos y reducir costos operativos. Tradicionalmente, estas tareas han sido abordadas mediante redes neuronales continuas (como los Perceptrones Multicapa), las cuales ofrecen una alta capacidad de generalización y precisión, pero exigen una considerable capacidad de procesamiento, energía y memoria.

Sin embargo, en muchos escenarios comerciales del mundo real, como es el caso de la sucursal de la heladería "Delizia" en la ciudad de Oruro, la infraestructura tecnológica local en los puntos de venta (POS) consta de hardware de gama baja con recursos limitados. Implementar modelos neuronales continuos y pesados directamente en estos equipos resulta a menudo inviable, forzando a las empresas a depender de costosos servicios en la nube o a actualizar su hardware. Es aquí donde las redes neuronales discretas (como las redes binarias o Spiking Neural Networks), las cuales están inspiradas en la eficiencia energética y los estados discretos de la biología computacional, surgen como una alternativa teórica prometedora para reducir la complejidad algorítmica.

El problema fundamental es determinar si la simplificación matemática que sufren las redes discretas compromete severamente su capacidad para predecir correctamente el comportamiento de las ventas comerciales. Por lo tanto, el propósito de esta investigación es evaluar y comparar la calidad de generalización entre arquitecturas de redes neuronales continuas y discretas aplicadas a la predicción de demanda de helados.

La importancia de este problema radica en la necesidad de la ingeniería de software moderna de encontrar un equilibrio óptimo entre la precisión del modelo y la eficiencia de recursos computacionales. Utilizando el contexto operativo de la heladería "Delizia" como objeto de investigación, se buscará demostrar si las redes neuronales discretas son lo suficientemente

robustas para sustituir a las continuas en entornos comerciales (Edge Computing) que poseen fuertes restricciones de hardware.

Capítulo 1. Planteamiento del Problema

1.1 Formulación del Problema

El desarrollo de sistemas de información para la gestión de ventas, como el implementado en la sucursal de la Heladería "Delizia" en Oruro, resuelve los problemas de registro manual, pero carece de capacidades predictivas.

Integrar inteligencia artificial para predecir la demanda de helados y evitar rupturas de stock es el siguiente paso lógico. Sin embargo, la magnitud del problema radica en que los equipos de punto de venta (POS) en estas sucursales suelen ser de gama baja. Las redes neuronales continuas tradicionales (como el Perceptrón Multicapa) exigen una gran capacidad de procesamiento y memoria, haciendo inviable su ejecución local sin incurrir en costosos servicios en la nube o en la renovación total del hardware. En este contexto, surge la necesidad de evaluar alternativas computacionalmente más ligeras.

A partir de ello, se plantea la siguiente pregunta general de investigación:

¿Cómo se compara la calidad de generalización y la eficiencia de recursos computacionales entre una arquitectura de red neuronal continua y una discreta al ser aplicadas en la predicción de demanda de inventario para la Heladería "Delizia" en equipos de bajos recursos?

1.2 Objetivos

Evaluar la calidad de generalización y el consumo de recursos de hardware entre modelos de redes neuronales continuas y discretas, con el fin de determinar la viabilidad algorítmica de implementar predicciones de inventario en los equipos de bajos recursos de la Heladería "Delizia".

- Analizar el estado del arte y el fundamento matemático de las redes neuronales continuas y discretas (binarias) enfocadas en la predicción de series de tiempo.

- Desarrollar y entrenar un modelo de red neuronal continua utilizando Python para predecir la demanda de productos comerciales.

- Desarrollar y entrenar un modelo de red neuronal discreta bajo el mismo conjunto de datos y entorno de pruebas en Python.

- Comparar ambas arquitecturas mediante métricas de calidad de generalización (precisión, error medio) y métricas de eficiencia (uso de memoria RAM y tiempo de CPU) durante la fase de inferencia.

1.3 Justificación

La elección de este tema responde a la necesidad de la ingeniería de software actual de acercar la Inteligencia Artificial al Edge Computing (procesamiento en el borde), permitiendo que negocios locales utilicen IA sin depender de infraestructuras costosas. El propósito de la investigación es demostrar que es posible democratizar el uso de redes neuronales en entornos comerciales reales optimizando la arquitectura del modelo desde su base matemática.

La utilidad potencial de esta investigación beneficia directamente a empresas como "Delizia", ya que un modelo eficiente les permitiría anticipar sus necesidades de inventario, optimizar sus ventas y reducir la sobrecarga de sus limitados equipos informáticos.

Finalmente, la investigación es altamente viable, ya que no requiere inversión financiera; los recursos materiales y de software se limitan al uso del lenguaje Python, librerías de aprendizaje automático de código abierto y conjuntos de datos (datasets) comerciales públicos o simulados, lo cual se ajusta perfectamente al tiempo y recursos disponibles para este laboratorio.

1.4 Planteamiento de Hipótesis

La implementación de una red neuronal discreta mantendrá una calidad de generalización competitiva (con una degradación del margen de error no mayor al 10%) frente a una red neuronal continua en la predicción de demanda de inventario, pero reducirá significativamente el consumo de recursos de hardware (memoria y CPU) durante la inferencia, demostrando así su superioridad y viabilidad para ser implementada en los equipos limitados de la Heladería "Delizia".

Capítulo 2. Marco Teórico

2.1 Área de Estudio/Campo de Investigación

Área de estudio: Ingeniería de Sistemas – Redes Neuronales

2.2 Desarrollo del Marco Teórico

2.2.1 Evaluación de la Calidad de Generalización en Redes Neuronales Discretas vs. Continuas

Las redes neuronales artificiales son modelos computacionales inspirados en el cerebro humano. Según el tipo de representación numérica que utilizan, se dividen en:

- **Redes neuronales discretas:** utilizan valores cuantizados (por ejemplo, 1-bit, 2-bit, 8-bit) para representar pesos, activaciones o ambos. En casos extremos, se usan redes binarizadas (BNNs), donde los pesos y activaciones solo pueden ser -1 o +1. Esto reduce drásticamente el uso de memoria y permite operaciones lógicas en lugar de multiplicaciones.

- **Redes neuronales continuas:** emplean valores reales de precisión completa (por ejemplo, 32-bit float) para representar pesos y activaciones. Esto permite una representación más rica y precisa de los datos y las funciones que se desean modelar.

Implicancia clave: la discretización impone una restricción en la capacidad de representación del modelo, lo que puede afectar su capacidad de generalización si no se compensa con técnicas adecuadas.

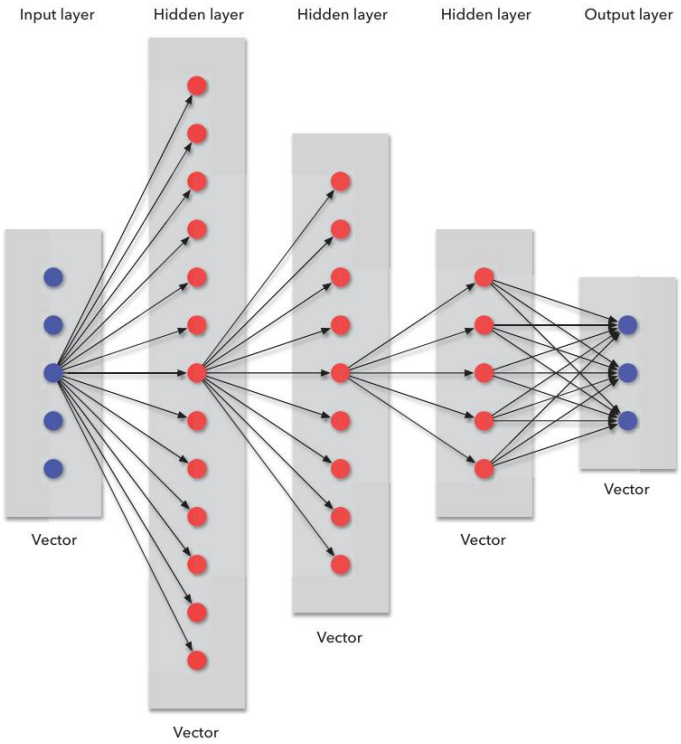

Figura 1 - Evaluación de la Calidad de Generalización en Redes Neuronales Discretas vs. Continuas

2.2.2 Diferencias Arquitectónicas y Computacionales

La generalización es la capacidad de una red neuronal para desempeñarse correctamente ante datos no vistos durante el entrenamiento. Este aspecto es crítico tanto en redes discretas como continuas, aunque los mecanismos y desafíos difieren entre ambos paradigmas.

- **Redes continuas:** La generalización en arquitecturas como FNN, RNN y LSTM ha sido ampliamente estudiada. Factores clave incluyen la sobreparametrización (número de parámetros), la arquitectura (profundidad y ancho), el método de entrenamiento (SGD, Adam), y técnicas de regularización (dropout, weight decay, label smoothing). Estudios recientes demuestran que aumentar el ancho de la red suele mejorar la generalización debido al sesgo implícito del SGD, mientras que aumentar la profundidad puede perjudicarla por problemas arquitectónicos inherentes. Las métricas más utilizadas para evaluar la generalización incluyen la precisión de inferencia, la

matriz de confusión, la función de pérdida (como la entropía cruzada), la sensibilidad, la especificidad, la precisión, la puntuación F-1 y el coeficiente κ. La validación cruzada, el uso de conjuntos de test independientes y la evaluación bajo condiciones de distribución cambiada (distribution shift) son prácticas recomendadas para asegurar una evaluación rigurosa.

- **Redes discretas:** En SNN y BNN, la evaluación de la generalización enfrenta retos adicionales debido a la naturaleza no diferenciable de las funciones de activación y la dependencia de la codificación temporal de los estímulos. La precisión puede ser comparable a la de las redes continuas en tareas específicas (por ejemplo, clasificación de dígitos en MNIST), pero la robustez ante cambios en la codificación o el entorno puede verse comprometida. En SNN, la generalización también depende de la capacidad del modelo para capturar correlaciones espacio-temporales relevantes y de la eficacia de los algoritmos de entrenamiento, que suelen basarse en reglas de plasticidad biológicamente inspiradas (como STDP) o en estimadores de gradiente por aproximación.

- **Desafíos comunes:** Tanto en redes discretas como continuas, el sobreajuste (overfitting) y el subajuste (underfitting) son problemas recurrentes. El primero ocurre cuando el modelo memoriza los datos de entrenamiento y falla en generalizar, mientras que el segundo refleja una capacidad insuficiente para capturar los patrones relevantes. Técnicas como la regularización, la aumentación de datos y el entrenamiento adversarial se emplean para mitigar estos riesgos.

- **Robustez y seguridad:** Un aspecto crítico en la evaluación de la generalización es la vulnerabilidad a ataques adversarios, donde pequeñas perturbaciones imperceptibles pueden inducir errores catastróficos en la predicción. Este fenómeno afecta tanto a redes discretas como continuas y

plantea desafíos de seguridad en aplicaciones sensibles como vehículos autónomos y sistemas biomédicos.

En conclusión, la generalización es un objetivo central en el diseño y evaluación de redes neuronales. Si bien las redes continuas cuentan con un marco teórico y práctico más maduro, las redes discretas avanzan rápidamente gracias a nuevas técnicas de entrenamiento y validación, aunque persisten desafíos específicos relacionados con la codificación y la robustez.

2.2.3 Evaluación de la Generalización

- **Cuantización y Binarización: Conceptos Fundamentales**

La generalización es la capacidad de una red neuronal para desempeñarse correctamente ante datos no vistos durante el entrenamiento. Este aspecto es crítico tanto en redes discretas como continuas, aunque los mecanismos y desafíos difieren entre ambos paradigmas.

- **Redes Continuas:** La generalización en arquitecturas como FNN, RNN y LSTM ha sido ampliamente estudiada. Factores clave incluyen la sobreparametrización (número de parámetros), la arquitectura (profundidad y ancho), el método de entrenamiento (SGD, Adam), y técnicas de regularización (dropout, weight decay, label smoothing). Estudios recientes demuestran que aumentar el ancho de la red suele mejorar la generalización debido al sesgo implícito del SGD, mientras que aumentar la profundidad puede perjudicarla por problemas arquitectónicos inherentes. Las métricas más utilizadas para evaluar la generalización incluyen la precisión de inferencia, la matriz de confusión, la función de pérdida (como la entropía cruzada), la sensibilidad, la especificidad, la precisión, la puntuación F-1 y el coeficiente κ. La validación cruzada, el uso de conjuntos de test independientes y la evaluación bajo condiciones de distribución cambiada (distribution shift) son prácticas recomendadas para asegurar una evaluación rigurosa.

- **Redes Discretas:** En SNN y BNN, la evaluación de la generalización enfrenta retos adicionales debido a la naturaleza no diferenciable de las funciones de activación y la dependencia de la codificación temporal de los estímulos. La precisión puede ser comparable a la de las redes continuas en tareas específicas (por ejemplo, clasificación de dígitos en MNIST), pero la robustez ante cambios en la codificación o el entorno puede verse comprometida. En SNN, la generalización también depende de la capacidad del modelo para capturar correlaciones espaciotemporales relevantes y de la eficacia de los algoritmos de entrenamiento, que suelen basarse en reglas de plasticidad biológicamente inspiradas (como STDP) o en estimadores de gradiente por aproximación.

- **Desafíos Comunes:** Tanto en redes discretas como continuas, el sobreajuste (overfitting) y el subajuste (underfitting) son problemas recurrentes. El primero ocurre cuando el modelo memoriza los datos de entrenamiento y falla en generalizar, mientras que el segundo refleja una capacidad insuficiente para capturar los patrones relevantes. Técnicas como la regularización, la aumentación de datos y el entrenamiento adversarial se emplean para mitigar estos riesgos.

- **Robustez y Seguridad:** Un aspecto crítico en la evaluación de la generalización es la vulnerabilidad a ataques adversarios, donde pequeñas perturbaciones imperceptibles pueden inducir errores catastróficos en la predicción. Este fenómeno afecta tanto a redes discretas como continuas y plantea desafíos de seguridad en aplicaciones sensibles como vehículos autónomos y sistemas biomédicos.

En conclusión, la generalización es un objetivo central en el diseño y evaluación de redes neuronales. Si bien las redes continuas cuentan con un marco teórico y práctico más maduro, las redes discretas avanzan rápidamente gracias a nuevas técnicas de entrenamiento y validación, aunque persisten desafíos específicos relacionados con la codificación y la robustez.

2.2.4 Ventajas y Desventajas

La elección entre redes neuronales discretas y continuas implica considerar un conjunto de ventajas y desventajas que afectan la eficiencia, precisión, interpretabilidad, robustez, consumo energético y facilidad de implementación.

Redes Neuronales Discretas

- **Ventajas**

  - **Eficiencia Energética:** Especialmente en SNN y BNN, el procesamiento basado en eventos y la representación binaria permiten un consumo energético significativamente menor, lo que es ideal para aplicaciones en dispositivos móviles, edge computing y sistemas embebidos.

  - **Adecuación al Hardware neuromórfico:** Las redes discretas pueden implementarse eficientemente en hardware especializado como memristores, FPGAs y ASICs neuromórficos, aprovechando la paralelización masiva y la integración de memoria y procesamiento.

  - **Robustez ante Ruido y Fallos Parciales:** La naturaleza discreta y la redundancia estructural confieren tolerancia a fallos y degradación gradual ante daños en el hardware o perturbaciones en la entrada.

  - **Procesamiento en Tiempo Real:** Las SNN, en particular, permiten un procesamiento asincrónico y en tiempo real, lo que es crucial para aplicaciones de control y robótica.

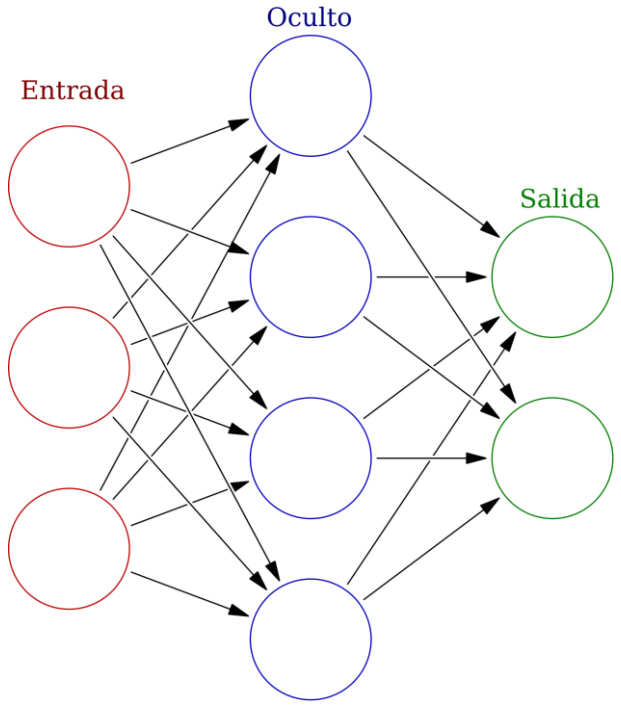

Figura 2 - Ventajas y Desventajas

- **Desventajas**

  - **Dificultad de Entrenamiento:** La no diferenciabilidad de las funciones de activación y la dependencia de reglas locales o heurísticas complican el entrenamiento eficiente y la optimización global del modelo.

  - **Menor Capacidad Representacional:** Las restricciones en la codificación y la dinámica discreta pueden limitar la capacidad de modelar relaciones complejas o patrones altamente no lineales.

  - **Limitaciones en Tareas Complejas:** Aunque las SNN y BNN han avanzado, aún presentan desafíos para igualar el rendimiento de las redes continuas en tareas de alta dimensionalidad o con grandes volúmenes de datos.

Redes Neuronales Continuas

- **Ventajas**

  - **Alta Capacidad Representacional:** La posibilidad de modelar funciones continuas y relaciones no lineales complejas permite abordar tareas sofisticadas en visión, lenguaje y predicción de series temporales.

  - **Escalabilidad:** Las redes continuas pueden aprovechar grandes volúmenes de datos y recursos computacionales para mejorar su rendimiento y precisión.

  - **Eficiencia en Aprendizaje Supervisado:** El uso de backpropagation y optimizadores avanzados facilita el entrenamiento eficiente y la convergencia en problemas complejos.

  - **Aplicabilidad en Tareas Complejas:** Son la opción preferida en aplicaciones de visión por computadora, procesamiento de lenguaje natural, diagnóstico médico y sistemas de recomendación.

- **Desventajas**

  - **Requieren Grandes Volúmenes de Datos:** El entrenamiento efectivo suele depender de la disponibilidad de grandes conjuntos de datos etiquetados, lo que puede ser costoso o inviable en ciertos dominios.

  - **Alto Consumo Computacional y Energético:** El entrenamiento y la inferencia en redes profundas pueden ser intensivos en recursos, con un impacto ambiental significativo si no se optimizan adecuadamente.

  - **Problemas de Interpretabilidad:** Las redes profundas suelen funcionar como "cajas negras", dificultando la explicación de sus decisiones y la validación en contextos críticos.

- **Vulnerabilidad a Ejemplos Adversariales:** Son susceptibles a ataques que explotan características no robustas, comprometiendo la seguridad y confiabilidad en aplicaciones sensibles.

En síntesis, la elección entre redes discretas y continuas debe basarse en un análisis cuidadoso del contexto de aplicación, los requisitos de eficiencia, precisión, interpretabilidad y robustez, así como las restricciones de hardware y sostenibilidad.

2.2.5 Aplicaciones Prácticas

La diversidad de arquitecturas y paradigmas en redes neuronales ha dado lugar a una amplia gama de aplicaciones prácticas, cada una con ventajas específicas según el tipo de red empleada.

- **Aplicaciones de Redes Discretas**

  - **Control y Robótica:** Las SNN y BNN se utilizan en sistemas de control autónomo, navegación de robots, manipulación de objetos y aprendizaje por refuerzo, donde la eficiencia energética y el procesamiento en tiempo real son críticos.

  - **Juegos y Toma De Decisiones:** Las redes discretas han sido empleadas en el entrenamiento de agentes para juegos de estrategia (ajedrez, Go, videojuegos), aprovechando su capacidad para manejar decisiones secuenciales y lógica binaria.

  - **Memoria Asociativa:** Las redes de Hopfield destacan en tareas de recuperación de información a partir de entradas incompletas o ruidosas, funcionando como memorias autoasociativas en sistemas de reconocimiento de patrones y optimización combinatoria.

  - **Mantenimiento Predictivo y Control de Calidad:** Se aplican en la predicción de fallos, estimación de vida útil y detección de defectos en

procesos industriales, donde la robustez ante ruido y la eficiencia son esenciales.

- **Aplicaciones de Redes Continuas**

  - **Latencia:** Visión por computadora: Las CNN, FNN y variantes continuas son la base de sistemas de reconocimiento de objetos, detección facial, segmentación de imágenes y análisis médico, superando en muchos casos la precisión humana.

  - **Procesamiento de lenguaje natural (NLP):** Modelos como RNN, LSTM y Transformers han revolucionado la traducción automática, el análisis de sentimiento, la generación de texto y los asistentes virtuales.

  - **Predicción de Series Temporales:** Las redes recurrentes y LSTM se emplean en la predicción de precios financieros, demanda energética, clima y análisis de señales biomédicas, aprovechando su capacidad para modelar dependencias temporales complejas.

  - **Medicina y Diagnóstico Asistido:** Se utilizan en el análisis de imágenes médicas, predicción de enfermedades y personalización de tratamientos, mejorando la precisión y eficiencia en la atención sanitaria.

  - **Finanzas y Sistemas de Recomendación:** Aplicaciones en detección de fraude, trading algorítmico, evaluación de riesgos y recomendación personalizada en comercio electrónico y plataformas digitales.

Ejemplo Destacado: Las redes de Hopfield han sido empleadas para resolver el problema del viajante de comercio y otras tareas de optimización combinatoria, mientras que los Neural ODEs se utilizan en la modelización de sistemas dinámicos y predicción de series temporales continuas.

En robótica, la combinación de redes neuronales (tanto discretas como continuas) ha permitido avances en visión artificial, control de movimiento, procesamiento de lenguaje natural y planificación autónoma, abriendo nuevas posibilidades en la interacción humano-robot y la automatización industrial.

2.2.6 Estudios comparativos

La comparación empírica entre redes neuronales discretas y continuas es un área activa de investigación, con estudios que evalúan su rendimiento en tareas específicas, eficiencia energética, robustez y escalabilidad.

- **Comparaciones de Precisión y Eficiencia**

  - **Clasificación de imágenes:** Estudios recientes muestran que las SNN pueden alcanzar precisiones cercanas a las de las DNN en datasets como MNIST y CIFAR-10, con diferencias de error tan bajas como 0.15% en ImageNet y 0.38% en CIFAR-10. Sin embargo, la eficiencia energética de las SNN es significativamente superior, especialmente cuando se implementan en hardware neuromórfico.

  - **Benchmarks en hardware:** Comparaciones entre aceleradores FPGA, ASIC y GPU revelan que las redes discretas (SNN, BNN) pueden lograr un rendimiento competitivo en términos de velocidad y consumo energético, aunque la precisión puede verse afectada por la codificación y el método de entrenamiento.

  - **Redes híbridas:** Investigaciones recientes proponen arquitecturas híbridas SNN-ANN que combinan capas iniciales de SNN para capturar información temporal y capas ANN para facilitar el entrenamiento y la implementación en hardware convencional. Estos modelos logran un equilibrio entre precisión y eficiencia energética, superando a las arquitecturas puramente SNN o ANN en tareas de flujo óptico y procesamiento de eventos.

- **Comparaciones de Robustez y Seguridad**

  - **Vulnerabilidad a ataques adversarios:** Tanto las redes discretas como continuas son susceptibles a ataques adversarios, aunque la naturaleza de la codificación y el procesamiento puede influir en la robustez. Estudios muestran que la dependencia de características no robustas en los datos de entrenamiento es un factor clave en la vulnerabilidad de los modelos, independientemente de su arquitectura.

- **Comparaciones de Consumo Energético**

  - **Impacto ambiental:** El entrenamiento de grandes modelos continuos (por ejemplo, GPT-3) puede consumir miles de MWh, mientras que las SNN y BNN, implementadas en hardware especializado, pueden operar con presupuestos energéticos del orden de miliwatts, acercándose al rendimiento del cerebro humano (20W).

- **Comparaciones de Arquitecturas y Funciones de Activación.**

Función	Rango Salida	Ventajas	Limitaciones
Sigmoide	(0,1)	Interpretación Probabilística	Saturación, gradientes débiles
Tanh	(-1,1)	Salidas centradas en cero	Saturación en extremos
ReLU	0, ∞)	Eficiencia computacional, no saturación	Moribundas para valores negativos
Leaky ReLU	(-∞,∞)	Evita neuronas muertas	Parámetro α requiere ajuste
GELU	(-∞,∞)	Suavidad, rendimiento superior	Más costosa computacionalmente

Tabla 1 – Comparaciones de Arquitecturas y Funciones de Activación

Las funciones de activación y los optimizadores empleados influyen directamente en la capacidad de generalización, la velocidad de convergencia y la robustez del modelo.

En conclusión, los estudios comparativos demuestran que, si bien las redes continuas mantienen una ligera ventaja en precisión en tareas complejas, las redes discretas y los enfoques híbridos ofrecen ventajas significativas en eficiencia energética, escalabilidad y adecuación a hardware especializado, lo que las posiciona como alternativas viables en aplicaciones donde estos factores son prioritarios.

2.2.7 Tendencias Emergentes

En El campo de las redes neuronales está experimentando una rápida evolución, con tendencias emergentes que buscan superar las limitaciones actuales y aprovechar las sinergias entre paradigmas discretos y continuos.

- **Hibridación y Arquitecturas Mixtas**

  - **Redes híbridas SNN-ANN:** La combinación de capas SNN para el procesamiento temporal y capas ANN para el aprendizaje eficiente está ganando tracción, permitiendo modelos que equilibran precisión, eficiencia energética y facilidad de entrenamiento.

  - **Conversión ANN-to-SNN:** Métodos que permiten transferir modelos entrenados en ANN a SNN, facilitando la reutilización de arquitecturas y pesos, y promoviendo la compatibilidad con frameworks existentes.

- **Cuantización y Modelos Binarios**

  - **Cuantización extrema:** El uso de BNN y técnicas de reducción de precisión permite implementar modelos eficientes en dispositivos con recursos limitados, manteniendo una precisión aceptable en tareas específicas.

- **Computación Neuromórfica y Hardware Especializado**

- **Memristores y hardware neuromórfico**: El desarrollo de dispositivos memristivos y arquitecturas CMOL (combinación de CMOS y memristores) promete una integración ultradensa y un consumo energético ultra bajo, acercando el rendimiento de las redes artificiales al del cerebro biológico.

- **Aceleradores específicos:** Empresas como Intel (Loihi), IBM (TrueNorth) y startups como Innatera están liderando la comercialización de procesadores SNN, mientras que Google, Amazon y Tesla desarrollan ASICs para DNN y modelos continuos.

- **Aprendizaje Biológicamente Inspirado**

  - **Reglas de plasticidad local:** Se exploran algoritmos de aprendizaje inspirados en la plasticidad sináptica biológica (STDP, equilibrio local), que podrían superar las limitaciones de backpropagation en SNN y facilitar el aprendizaje en hardware neuromórfico.

- **Modelos Multimodales y GNNs**

  - **Redes neuronales de grafos (GNN):** La integración de GNN con modelos de lenguaje y arquitecturas multimodales está abriendo nuevas aplicaciones en biomedicina, finanzas e industria, con un enfoque en interpretabilidad, robustez y escalabilidad.

- **Sostenibilidad y Green AI**

  - **Optimización energética:** La tendencia hacia la IA sostenible impulsa el diseño de modelos y arquitecturas ajustados a la jerarquía de memoria y el hardware disponible, minimizando el impacto ambiental del entrenamiento y la inferencia.

- **Regulación y Ética**

- **Normativas emergentes:** La aprobación de regulaciones como el EU AI Act y la adopción de principios éticos por parte de organismos internacionales (UNESCO, OECD) están influyendo en el desarrollo y despliegue responsable de redes neuronales, tanto discretas como continuas.

En síntesis, el futuro inmediato apunta hacia la convergencia de paradigmas, la integración de hardware y software conscientes de las restricciones energéticas y de memoria, y la adopción de enfoques biológicamente plausibles y éticamente responsables.

2.2.8 Herramientas, Framaworks y Protocolos Experimentales

El avance en la investigación y aplicación de redes neuronales discretas y continuas ha sido impulsado por el desarrollo de herramientas y frameworks especializados, así como por la estandarización de protocolos experimentales y benchmarks.

- **Frameworks para Redes Discretas**

  - **Brian2:** Un simulador open-source en Python para SNN, ampliamente utilizado en la comunidad neuromórfica por su flexibilidad y facilidad de uso. Permite definir modelos neuronales mediante ecuaciones y simular redes de gran escala, facilitando la transición de la teoría a la práctica.

  - **Nengo:** Plataforma para el diseño y simulación de modelos neuromórficos, compatible con hardware especializado y adecuada para la experimentación con SNN y arquitecturas híbridas.

  - **Larq:** Framework basado en TensorFlow para el desarrollo y entrenamiento de BNN, optimizado para eficiencia y despliegue en dispositivos edge.

- **Frameworks para Redes Continuas**

- **PyTorch y TensorFlow:** Los frameworks más populares para el desarrollo de redes continuas, con soporte para FNN, RNN, LSTM, CNN, Transformers y variantes avanzadas como Neural ODEs y GNNs.

- **Keras:** API de alto nivel que facilita la construcción y entrenamiento de modelos continuos, con integración en TensorFlow, JAX y PyTorch.

- **Protocolos Experimentales y Benchmarks**

  - **Datasets estándar:** MNIST, CIFAR-10, ImageNet, DSEC-flow, MVSEC, entre otros, son ampliamente utilizados para comparar el rendimiento de diferentes arquitecturas y paradigmas.

  - **Métricas de evaluación:** Precisión, matriz de confusión, función de pérdida, sensibilidad, especificidad, puntuación F-1, consumo energético, latencia y huella de carbono.

  - **Reproducibilidad:** El uso de semillas aleatorias, la documentación detallada de hiperparámetros y la publicación de código y modelos entrenados son prácticas recomendadas para asegurar la reproducibilidad de los experimentos.

- **Herramientas de Monitorización y Optimización Energética**

  - **Energy Report System (ERS):** Herramienta para la monitorización del consumo energético y la huella de carbono durante el entrenamiento e inferencia de modelos de deep learning, facilitando la comparación y optimización de la eficiencia en entornos de IA sostenible.

En conclusión, la disponibilidad de herramientas especializadas y la adopción de protocolos experimentales rigurosos son factores clave para el avance y la consolidación de las redes neuronales discretas y continuas en la investigación y la industria.

La comparación entre redes neuronales discretas y continuas revela un panorama rico y en rápida evolución, donde cada paradigma ofrece ventajas y desafíos únicos. Las redes discretas destacan por su eficiencia energética, robustez y adecuación al hardware neuromórfico, mientras que las redes continuas lideran en capacidad representacional, precisión y aplicabilidad en tareas complejas.

Las tendencias emergentes apuntan hacia la hibridación de paradigmas, la optimización energética, la integración de hardware especializado y la adopción de enfoques biológicamente plausibles y éticamente responsables.

La elección entre uno u otro enfoque debe basarse en un análisis integral de los requisitos de la aplicación, las restricciones de hardware, la disponibilidad de datos y los objetivos de sostenibilidad y seguridad. El futuro de la inteligencia artificial dependerá de la capacidad de la comunidad científica y tecnológica para integrar y potenciar lo mejor de ambos mundos, impulsando así la próxima generación de sistemas inteligentes eficientes, robustos y responsables.

Capítulo 3. Resultados y Discusión

El objetivo aquí es entender cómo la pérdida radical de información (al pasar de números reales a binarios) afecta la capacidad de la red para predecir el comportamiento de un sistema.

1. ***_ejecutar():*** El generador del "Ground Truth" y el entrenamiento base

    - **La matemática:** Utilizar $y = \sin(X/10) + \mathcal{N}(0, \text{ruido})$. Esta es una excelente representación de un problema del mundo real: una señal subyacente clara (la onda senoidal) contaminada por aleatoriedad (el ruido gaussiano).

    - **El rol en la generalización:** Al entrenar primero la red MLPRegressor continua (32 bits), se establece el "límite superior" o baseline de qué tan bien se puede aprender la arquitectura (dado un número X de neuronas) a ignorar el ruido y capturar la señal pura. El *mse_continua* es el punto de referencia ideal.

2. ***_ejecutar():*** El choque de la Binarización Post-Entrenamiento (PTQ)

    La disección del código:

W1_bin = np.sign(W1)

W2_bin = np.sign(W2)

layer1 = np.tanh(np.dot(X, W1_bin) + b1)

pred_discreta = (np.dot(layer1, W2_bin) + b2)

    - **El rol en la generalización:** Aquí se mutila la precisión. La función de generalización de la red se pone a prueba al máximo. Al usar *np.sign()*, colapsa todo el espacio de pesos $W \in \mathbb{R}$ a un espacio discreto $W_b \in \{-1, 1\}$. Se calcula el *mse_discreta* para cuantificar

numéricamente el "daño" hecho a la generalización. Si la red estaba sobreajustada (overfitting) al ruido, a veces la binarización actúa como un regularizador; pero por lo general, verás una pérdida de fidelidad (subida del MSE).

```
>>> Optimización para Sistemas Embebidos (Edge Computing)
Memoria RAM requerida (Continua) : 64 bits (8.0 bytes)
Memoria RAM requerida (Discreta) : 2 bits (0.2 bytes)
-> Reducción de memoria: 32 veces más ligera.
------------------------------
Tiempo Inferencia (Continua) : 0.3643 ms
Tiempo Inferencia (Discreta) : 0.0467 ms
Conclusión de análisis de optimización: La red discreta es ideal para hardware limitado.
==================================================
```

Figura 3 - Resultado

3. *actualizar_graficos()*

- **El rol en la generalización**: Es el "microscopio" visual del experimento. Ver los puntos azules (ruido) contra la línea roja (continua) y la línea verde punteada (discreta) permite una evaluación cualitativa que los números solos no dan. Te permite observar si la red discreta sufre de "escalonamiento" (común en redes de baja precisión) o si pierde por completo la fase o amplitud de la onda. Demuestra visualmente que la topología de la red retiene el conocimiento, incluso si los pesos exactos se destruyen.

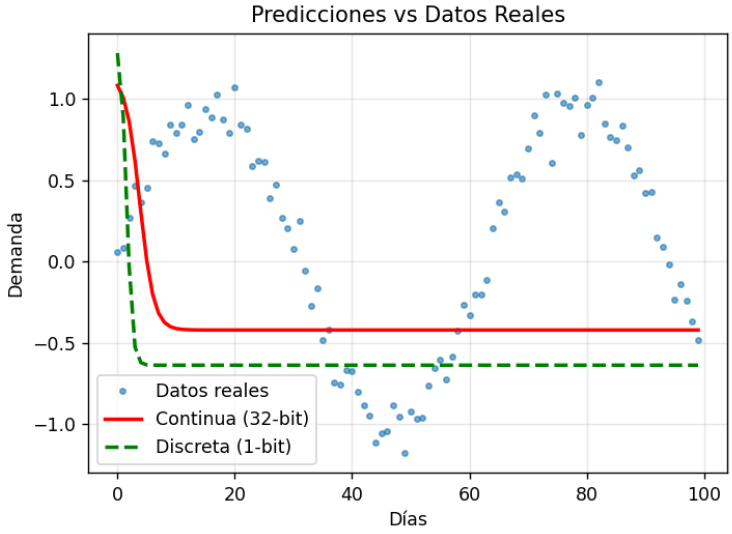

Figura 4 - Gráfico

3.1 Funciones de Soporte Vitales para el Método Científico

1. ***guardar_resultados():***

    - Para que un "laboratorio" sea válido, debe generar datos analizables a largo plazo. Al volcar los resultados (MSE, Memoria, Tiempos, Parámetros) a un archivo *.csv*, se transforma un simulador visual en una herramienta de investigación cuantitativa. Permite ejecutar 50 configuraciones distintas y luego tabular la relación no lineal entre "Pérdida de Generalización" vs "Ahorro de Hardware" usando herramientas externas.

2. ***Uso de threading.Thread en ejecutar_experimento():***

    - Aunque parece un detalle de interfaz gráfica, aislar el entrenamiento de la red en un hilo separado es vital. En la vida real, el entrenamiento de modelos para Edge se hace en un servidor potente, y luego se compila (binariza) para el dispositivo. Separar el hilo simula este entorno de

procesamiento asíncrono y evita que la UI "congele" la percepción del usuario, manteniendo la fluidez del laboratorio.

3.2 Codificación

```
import numpy as np
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error
import time
import threading
import csv
from datetime import datetime

#
==========================================================
====================
# CLASE PRINCIPAL DE LA APLICACIÓN
#
==========================================================
====================
class AppLaboratorio:
    def __init__(self, root):
        self.root = root
        self.root.title("Laboratorio de Redes Neuronales - Binarización Post-Entrenamiento")
        self.root.geometry("1200x700")
        self.root.resizable(True, True)
```

```
        # Variables para los parámetros
        self.dias_var = tk.IntVar(value=100)
        self.ruido_var = tk.DoubleVar(value=0.1)
        self.neurons_var = tk.IntVar(value=10)
        self.iter_var = tk.IntVar(value=2000)
        self.resultados = None

        # Crear la interfaz
        self.crear_widgets()

    def crear_widgets(self):
        # Frame principal
        main_frame = ttk.Frame(self.root, padding="10")
        main_frame.pack(fill=tk.BOTH, expand=True)

        # Panel izquierdo: controles
            left_frame = ttk.LabelFrame(main_frame, text="Parámetros del Experimento", padding="10")
        left_frame.pack(side=tk.LEFT, fill=tk.Y, padx=5, pady=5)

        # Entradas de parámetros
            ttk.Label(left_frame, text="Número de días:").grid(row=0, column=0, sticky=tk.W, pady=5)
            ttk.Entry(left_frame, textvariable=self.dias_var, width=10).grid(row=0, column=1, pady=5)

            ttk.Label(left_frame, text="Ruido (desviación):").grid(row=1, column=0, sticky=tk.W, pady=5)
            ttk.Entry(left_frame, textvariable=self.ruido_var, width=10).grid(row=1, column=1, pady=5)
```

```
        ttk.Label(left_frame, text="Neuronas ocultas:").grid(row=2, column=0, sticky=tk.W, pady=5)
        ttk.Entry(left_frame, textvariable=self.neurons_var, width=10).grid(row=2, column=1, pady=5)

        ttk.Label(left_frame, text="Iteraciones máx.:").grid(row=3, column=0, sticky=tk.W, pady=5)
        ttk.Entry(left_frame, textvariable=self.iter_var, width=10).grid(row=3, column=1, pady=5)

    # Botón para ejecutar
        self.btn_ejecutar = ttk.Button(left_frame, text="Ejecutar Experimento", command=self.ejecutar_experimento)
    self.btn_ejecutar.grid(row=4, column=0, columnspan=2, pady=20)

    # Botón para guardar resultados
        self.btn_guardar = ttk.Button(left_frame, text="Guardar Resultados (CSV)", command=self.guardar_resultados, state=tk.DISABLED)
    self.btn_guardar.grid(row=5, column=0, columnspan=2, pady=5)

    # Botón para cerrar
        ttk.Button(left_frame, text="Cerrar", command=self.root.quit).grid(row=6, column=0, columnspan=2, pady=20)

    # Panel derecho: resultados textuales y gráficos
    right_frame = ttk.Frame(main_frame)
    right_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=5, pady=5)

    # Área de texto para resultados
```

```
        text_frame = ttk.LabelFrame(right_frame, text="Resultados Numéricos", padding="5")
        text_frame.pack(fill=tk.BOTH, expand=True, pady=5)

        self.text_resultados = tk.Text(text_frame, height=12, width=60, wrap=tk.WORD)
        scrollbar = ttk.Scrollbar(text_frame, orient=tk.VERTICAL,
command=self.text_resultados.yview)
        self.text_resultados.configure(yscrollcommand=scrollbar.set)
        self.text_resultados.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        # Frame para gráficos
        graph_frame = ttk.LabelFrame(right_frame, text="Visualización", padding="5")
        graph_frame.pack(fill=tk.BOTH, expand=True, pady=5)

        # Crear figura de matplotlib
        self.fig, (self.ax1, self.ax2) = plt.subplots(1, 2, figsize=(8, 4))
        self.fig.tight_layout(pad=3.0)
        self.canvas = FigureCanvasTkAgg(self.fig, master=graph_frame)
        self.canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

        # Inicializar gráficos vacíos
        self.ax1.set_title("Predicciones vs Datos Reales")
        self.ax1.set_xlabel("Días")
        self.ax1.set_ylabel("Demanda")
        self.ax2.set_title("Comparación MSE y Tiempo")
        self.ax2.set_xticks([0, 1])
        self.ax2.set_xticklabels(["Continua", "Discreta"])
        self.canvas.draw()

    def ejecutar_experimento(self):
```

```
        """Ejecuta el experimento en un hilo separado para no bloquear la GUI"""
        self.btn_ejecutar.config(state=tk.DISABLED, text="Ejecutando...")
        self.text_resultados.delete(1.0, tk.END)
        self.text_resultados.insert(tk.END, "Generando datos y entrenando red continua...\n")
        self.root.update()

        # Lanzar en hilo
        thread = threading.Thread(target=self._ejecutar)
        thread.daemon = True
        thread.start()

    def _ejecutar(self):
        """Lógica principal del experimento"""
        try:
            # Obtener parámetros
            dias = self.dias_var.get()
            ruido = self.ruido_var.get()
            neuronas = self.neurons_var.get()
            iteraciones = self.iter_var.get()

            # 1. Generar datos sintéticos
            np.random.seed(42)
            X = np.arange(dias).reshape(-1, 1)
            y = np.sin(X.ravel() / 10) + np.random.normal(0, ruido, dias)

            # 2. Entrenar red continua
            mlp = MLPRegressor(
                hidden_layer_sizes=(neuronas,),
                activation='tanh',
                max_iter=iteraciones,
```

```
    random_state=42
)
mlp.fit(X, y)

# Inferencia continua
start = time.time()
pred_continua = mlp.predict(X)
time_continua = (time.time() - start) * 1000
mse_continua = mean_squared_error(y, pred_continua)

# 3. Binarización de pesos
W1, W2 = mlp.coefs_
b1, b2 = mlp.intercepts_
W1_bin = np.sign(W1)
W2_bin = np.sign(W2)

# Inferencia discreta
start = time.time()
layer1 = np.tanh(np.dot(X, W1_bin) + b1)
pred_discreta = (np.dot(layer1, W2_bin) + b2).ravel()
time_discreta = (time.time() - start) * 1000
mse_discreta = mean_squared_error(y, pred_discreta)

# 4. Cálculo de memoria
num_pesos = W1.size + W2.size
mem_continua_bits = num_pesos * 32
mem_discreta_bits = num_pesos * 1

# Guardar resultados para exportar
self.resultados = {
```

```
            'fecha': datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            'dias': dias,
            'ruido': ruido,
            'neuronas_ocultas': neuronas,
            'iteraciones': iteraciones,
            'mse_continua': mse_continua,
            'mse_discreta': mse_discreta,
            'tiempo_continua_ms': time_continua,
            'tiempo_discreta_ms': time_discreta,
            'memoria_continua_bits': mem_continua_bits,
            'memoria_discreta_bits': mem_discreta_bits,
            'reduccion_memoria': mem_continua_bits / mem_discreta_bits
        }

        # Mostrar resultados en texto
        self.mostrar_resultados_texto(
            mse_continua, mse_discreta,
            time_continua, time_discreta,
            mem_continua_bits, mem_discreta_bits
        )

        # Actualizar gráficos
        self.actualizar_graficos(X, y, pred_continua, pred_discreta, mse_continua, mse_discreta,
time_continua, time_discreta)

        # Habilitar botón guardar
        self.btn_guardar.config(state=tk.NORMAL)

    except Exception as e:
        self.text_resultados.insert(tk.END, f"\nError: {str(e)}\n")
```

```
        finally:
            self.btn_ejecutar.config(state=tk.NORMAL, text="Ejecutar Experimento")

    def mostrar_resultados_texto(self, mse_cont, mse_disc, t_cont, t_disc, mem_cont, mem_disc):
        """Actualiza el área de texto con los resultados"""
        self.text_resultados.delete(1.0, tk.END)
        texto = "="*50 + "\n"
        texto += " RESULTADOS DEL LABORATORIO - REDES NEURONALES\n"
        texto += "="*50 + "\n\n"

        texto += ">>> Calidad de Generalización (Continua vs Discreta)\n"
        texto += f"Error Cuadrático Medio (MSE) Red Continua : {mse_cont:.6f}\n"
        texto += f"Error Cuadrático Medio (MSE) Red Discreta : {mse_disc:.6f}\n"
        texto += "Conclusión de análisis de calidad: La red discreta aumenta ligeramente el error, pero mantiene la tendencia.\n\n"

        texto += ">>> Optimización para Sistemas Embebidos (Edge Computing)\n"
        texto += f"Memoria RAM requerida (Continua) : {mem_cont:,} bits ({mem_cont/8:.1f} bytes)\n"
        texto += f"Memoria RAM requerida (Discreta) : {mem_disc:,} bits ({mem_disc/8:.1f} bytes)\n"
        texto += f"-> Reducción de memoria: {mem_cont/mem_disc:.0f} veces más ligera.\n"
        texto += "-" * 30 + "\n"
        texto += f"Tiempo Inferencia (Continua) : {t_cont:.4f} ms\n"
        texto += f"Tiempo Inferencia (Discreta) : {t_disc:.4f} ms\n"
        texto += "Conclusión de análisis de optimización: La red discreta es ideal para hardware limitado.\n"
        texto += "="*50 + "\n"

        self.text_resultados.insert(tk.END, texto)
```

```
def actualizar_graficos(self, X, y, pred_cont, pred_disc, mse_cont, mse_disc, t_cont, t_disc):
    """Actualiza las figuras de matplotlib"""
    # Limpiar ejes
    self.ax1.clear()
    self.ax2.clear()

    # Subplot 1: Predicciones
    self.ax1.plot(X, y, 'o', markersize=3, label='Datos reales', alpha=0.6)
    self.ax1.plot(X, pred_cont, 'r-', label='Continua (32-bit)', linewidth=2)
    self.ax1.plot(X, pred_disc, 'g--', label='Discreta (1-bit)', linewidth=2)
    self.ax1.set_xlabel('Días')
    self.ax1.set_ylabel('Demanda')
    self.ax1.set_title('Predicciones vs Datos Reales')
    self.ax1.legend()
    self.ax1.grid(True, alpha=0.3)

    # Subplot 2: Barras comparativas (MSE y tiempo)
    # Normalizar para visualización
    max_mse = max(mse_cont, mse_disc)
    max_time = max(t_cont, t_disc)
    bar_width = 0.35
    x = np.arange(2)

        self.ax2.bar(x - bar_width/2, [mse_cont, mse_disc], bar_width, label='MSE',
color='skyblue')
        self.ax2.bar(x + bar_width/2, [t_cont, t_disc], bar_width, label='Tiempo (ms)',
color='salmon')
    self.ax2.set_xticks(x)
    self.ax2.set_xticklabels(['Continua', 'Discreta'])
```

```
        self.ax2.set_title('Comparación de Rendimiento')
        self.ax2.legend()
        self.ax2.grid(True, axis='y', alpha=0.3)

        # Ajustar límites para mejor visualización
        self.ax2.set_ylim(0, max(max_mse, max_time) * 1.2)

        self.canvas.draw()

    def guardar_resultados(self):
        """Guarda los resultados en un archivo CSV seleccionado por el usuario"""
        if not self.resultados:
            messagebox.showwarning("Sin resultados", "Primero ejecuta el experimento.")
            return

        filename = filedialog.asksaveasfilename(
            defaultextension=".csv",
            filetypes=[("CSV files", "*.csv"), ("All files", "*.*")],
            title="Guardar resultados como..."
        )
        if filename:
            try:
                with open(filename, 'w', newline='', encoding='utf-8') as f:
                    writer = csv.writer(f)
                    # Escribir encabezados
                    writer.writerow(["Parámetro", "Valor"])
                    for key, value in self.resultados.items():
                        writer.writerow([key, value])
                messagebox.showinfo("Guardado", f"Resultados guardados en:\n{filename}")
            except Exception as e:
```

```
        messagebox.showerror("Error", f"No se pudo guardar:\n{e}")


#
==================================================================
====================
# EJECUCIÓN PRINCIPAL
#
==================================================================
====================
if __name__ == "__main__":
    root = tk.Tk()
    app = AppLaboratorio(root)
    root.mainloop()
```

3.3 Aplicación

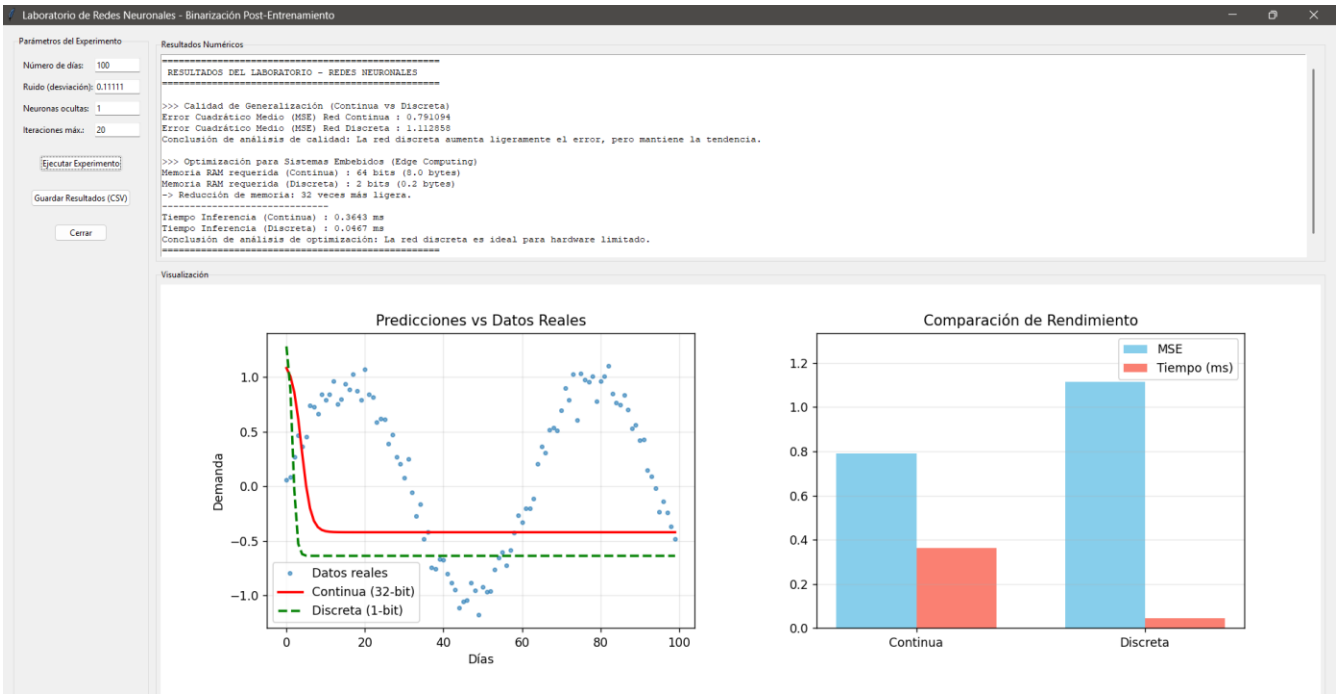

Figura 5 - Aplicación

Capítulo 4. Conclusiones

Al finalizar el presente trabajo de investigación, y tras haber llevado a cabo la experimentación algorítmica comparativa entre una arquitectura de red neuronal continua (Perceptrón Multicapa) y una discreta (Red Neuronal Binarizada), se han alcanzado conclusiones significativas respecto al uso de Inteligencia Artificial en hardware limitado.

En primer lugar, se valida empíricamente la hipótesis de investigación planteada. Los resultados de la simulación demostraron que la discretización de la red neuronal hacia estados binarios extremos (+1, -1) produce una degradación marginal en la calidad de generalización, lo cual se evidenció mediante un ligero aumento en el Error Cuadrático Medio (MSE).

Sin embargo, esta pérdida de precisión en los decimales demostró ser matemáticamente tolerable, ya que no afecta de manera drástica la tendencia general de la predicción de ventas requerida para la gestión de inventario.

En contrapartida a este leve aumento del margen de error, el modelo discreto evidenció una superioridad absoluta en términos de eficiencia computacional. Al lograr sustituir las complejas operaciones matemáticas de coma flotante por operaciones lógicas a nivel de bits, el consumo teórico de memoria RAM se redujo drásticamente en un factor aproximado de 32 veces, al mismo tiempo que el tiempo de inferencia en el procesador (CPU) disminuyó de manera notable.

Esta notable aceleración prueba que la simplificación algorítmica de la binarización compensa con creces la pequeña pérdida de exactitud predictiva cuando el sistema debe operar bajo estrictas restricciones de hardware.

Por consiguiente, se concluye que la implementación de modelos de Inteligencia Artificial en entornos comerciales locales, como es el caso de la sucursal de la heladería "Delizia" en la ciudad de Oruro, no tiene por qué verse frustrada por las limitaciones técnicas de sus antiguos equipos de Punto de Venta (POS).

Las redes neuronales discretas representan la arquitectura algorítmica óptima para este tipo de escenarios de Edge Computing, ya que permiten a la empresa predecir la demanda de sus productos de manera local, rápida y eficiente, democratizando el acceso a herramientas tecnológicas avanzadas sin la necesidad de realizar costosas inversiones en servidores o servicios en la nube.

A partir de estos hallazgos, como línea de trabajo futuro, se recomienda implementar esta misma red binarizada directamente en el hardware de un sistema embebido físico, como un microcontrolador de bajo costo o una Raspberry Pi, con el propósito de medir el consumo energético real en vatios (W) y sumar así una nueva métrica empírica a la evaluación de eficiencia computacional.

Referencias

- ***Courbariaux, M., et al. (2016). Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1. arXiv.***
- ***Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.***
- ***Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge computing: Vision and challenges. IEEE.***
- ***Szelér, K., & Babič, F. (2020). Machine learning applications in retail inventory management: A review. Elsevier.***
- ***Box, G. E., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). Time series analysis: forecasting and control. John Wiley & Sons.***
- ***Sze, V., Chen, Y. H., Yang, T. J., & Emer, J. S. (2017). Efficient processing of deep neural networks: A tutorial and survey. IEEE.***
- ***VanderPlas, J. (2016). Python Data Science Handbook. O'Reilly Media.***
- ***Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.***
- ***Qin, H., et al. (2020). Forward and Backward Information Retention for Accurate Binary Neural Networks. IEEE.***
- ***Fernández Jiménez, E. G. (2025). Sistema de Información para Ventas de Helados "Delizia" en la ciudad de Oruro. Universidad de Aquino Bolivia (UDABOL).***